

Approval SLAs and Escalation Latency, Worked Out by Hand

How a four-rung escalation ladder turns “a human might never answer” into a bounded, computable expected wait — one refund approval, counted by hand.

What this document does. It takes the one governance mechanism from Module 4.7 — the *refund-agent*’s four-rung escalation ladder ($L1 \rightarrow L2 \rightarrow L3 \rightarrow L4$) — fixes a per-rung answer probability and an SLA for each rung, and then computes, from first principles, the probability a request is resolved at each rung, the *expected approval latency*, the fraction auto-escalated past L1, and the steady-state queue depth via Little’s law. Nothing is hand-waved. Each number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes a ladder of SLAs (L1 15 min, then escalate); we rebuild the *wait that ladder actually produces*.

Where this sits. This is **Topic 1 of Module 4.7** (Human-in-the-Loop & Governance) — the number behind the “auto-escalate on SLA breach” failsafe. Its companions: Module 4.1 (the checkpointer + cron that pauses and promotes paused threads) and Module 4.3 (circuit breakers, whose trips drop straight into L2 of this same ladder).

Concepts covered, in order: the escalation ladder as a probability tree $\cdot P(\text{reach rung}) = \prod (\text{prior misses}) \cdot P(\text{resolved at rung}) \cdot \text{expected latency} = \sum P(\text{reach}) \cdot E[\text{wait}] \cdot \text{fraction auto-escalated} \cdot \text{worst-case bound} = \sum \text{SLAs} \cdot \text{reviewer load} = \lambda \cdot P(\text{needs approval}) \cdot \text{Little’s law queue depth } L = \lambda W \cdot \text{the expedited path that lets gates scale sublinearly}.$

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one approval climbing a four-rung ladder

An APPROVE-tier action (the module’s example: `issue refund $186`) pauses the agent and waits for a human. A human is not guaranteed to answer. So the run does not sit forever: each *rung* of the ladder has an SLA — a maximum time it may wait — and if that rung does not answer within its SLA, a cron job auto-escalates the paused thread to the next rung. After the last rung lapses, the run fails soft. The whole mechanism is a probability tree: at each rung the request is either answered (and done) or times out (and climbs).

The quantities. We give each rung an SLA and a probability that *that* rung answers *given the request reached it*. These are illustrative but round, so the tree is followable on paper. The mechanics are identical at any values — only the constants change.

Rung	Owner (from the module)	SLA	Answer prob. a_i
L1	First-line support agent	15 min	0.70
L2	Support lead	30 min	0.60
L3	Ops manager	60 min (1 h)	0.50
L4	Compliance / on-call	240 min (4 h)	1.00 (eventual)

Notation. a_i is the probability rung i answers *given it was reached*. $r_i = P(\text{reach rung } i)$. A

request resolved *at* rung i has waited the full SLA of every rung below it (each timed out) plus, on average, half of rung i 's own window. We set $a_4 = 1$: L4 always resolves (answer or fail-soft), so the tree terminates.

Intuition

The ladder is not a queue of people; it is a sequence of *deadlines*. Each rung gets one chance to answer within its window. Most requests die young — L1 catches 70% of them in fifteen minutes. The few that survive carry the full timeout of every rung they outlived, so the rare deep escalation is expensive in wall-clock time even though it is cheap in probability. The expected wait is a weighted average where the heavy weights sit on the cheap, shallow rungs.

2 Step 1 — probability of reaching each rung

A request reaches rung i only if *every* rung below it failed to answer in time. Reaching rung 1 is certain ($r_1 = 1$). Reaching rung i requires missing rungs $1 \dots i-1$:

$$r_i = \prod_{j < i} (1 - a_j), \quad P(\text{resolved at } i) = r_i a_i. \quad (1)$$

Working the four rungs: $r_1 = 1$. Miss L1 with prob $1 - 0.7 = 0.3$, so $r_2 = 0.3$. Miss L2 too: $r_3 = 0.3 \cdot (1 - 0.6) = 0.3 \cdot 0.4 = 0.12$. Miss L3 as well: $r_4 = 0.12 \cdot (1 - 0.5) = 0.12 \cdot 0.5 = 0.06$.

Rung	SLA	a_i	$r_i = P(\text{reach})$	$P(\text{resolved at } i) = r_i a_i$
L1	15	0.70	1.0000	0.7000
L2	30	0.60	0.3000	0.1800
L3	60	0.50	0.1200	0.0600
L4	240	1.00	0.0600	0.0600
Total resolved				1.0000

The resolution probabilities sum to exactly 1. ✓ Every request resolves somewhere — it must, because $a_4 = 1$ closes the tree. Note the **fraction auto-escalated past L1** is simply $r_2 = 1 - a_1 = 0.30$: three in ten requests are too slow for the first-line reviewer and climb at least once.

Mini-example — this operation in isolation

Read the column r_i as a survival curve. Starting from 1.00, each rung multiplies by its miss-probability: $\times 0.3$, then $\times 0.4$, then $\times 0.5$. By L4 only 6% of requests remain. This is why paging compliance “only at L4” (the module’s page policy) is affordable: the page fires on 6% of approvals, not all of them. Cut a_1 from 0.70 to 0.50 and r_2 jumps to 0.50 — suddenly half your traffic escalates, and L2 drowns. The shallow rung’s hit rate is the load-bearing number for everything above it.

3 Step 2 — the wait booked at each rung

A request resolved *at* rung i did not get there for free: every rung below it burned its *entire* SLA (that is what “timed out” means), and then rung i answered somewhere inside its own window — on average, halfway. So the wait conditional on resolving at rung i is

$$w_i = \underbrace{\sum_{j < i} \text{SLA}_j}_{\text{full timeouts burned}} + \underbrace{\frac{1}{2} \text{SLA}_i}_{\text{partial wait at this rung}}. \quad (2)$$

Rung	prior SLAs	$\sum_{j < i} \text{SLA}_j$	$\frac{1}{2} \text{SLA}_i$	w_i (min)
L1	0		7.5	7.5
L2	15		15.0	30.0
L3	45		30.0	75.0
L4	105		120.0	225.0

Sample check, L3: prior SLAs = 15 + 30 = 45; half of L3’s 60-min window = 30; sum = 75. ✓ A request that finally gets a manager has already burned the L1 and L2 windows in full and then waited, on average, half an hour into the manager’s window.

Watch out

The half-SLA term assumes a resolution *at* a rung lands uniformly inside that rung’s window. That is a modelling choice, not a law. If your reviewers tend to answer right at the buzzer (very common — humans work to deadlines), the true w_i skews toward the *full* SLA, not half. The *structure* of Equation (2) — prior timeouts in full, this rung in part — is invariant; the “half” is the one soft number here, and it is the first thing to replace with your own measured distribution.

4 Step 3 — expected approval latency

Expected latency is the wait at each rung, weighted by the probability of resolving there — exactly the law of total expectation over the probability tree:

$$\mathbb{E}[\text{latency}] = \sum_i P(\text{resolved at } i) \cdot w_i = \sum_i r_i a_i w_i \quad (3)$$

Plugging in both columns we just built:

$$\mathbb{E}[\text{latency}] = 0.70(7.5) + 0.18(30) + 0.06(75) + 0.06(225) = 5.25 + 5.40 + 4.50 + 13.50.$$

$$\mathbb{E}[\text{latency}] = \mathbf{28.65 \text{ min.}} \quad \checkmark$$

So the *average* refund approval clears in under half an hour, even though the ladder’s *worst case* is much larger. That worst case is the sum of all four SLAs — the longest a single request can possibly wait before fail-soft:

$$\text{worst-case wait} = \sum_i \text{SLA}_i = 15 + 30 + 60 + 240 = \mathbf{345} \text{ min} = 5.75 \text{ h.}$$

Intuition

Two numbers, two jobs. The **expected** latency (28.65 min) is what you put in an SLA promise to the business — it is dominated by the cheap shallow rungs where most traffic resolves. The **worst-case** bound (5.75 h) is what you put in the customer-facing fail-soft copy and the on-call runbook — it is set entirely by the SLAs, independent of the answer probabilities. Auto-escalation is what makes the worst case *finite*: without it, a silent reviewer means an unbounded wait. The ladder trades “maybe never” for “at most 5.75 hours, then a graceful fallback.”

5 Step 4 — reviewer load, batching, and queue depth

Latency is the experience of one request. *Load* is the experience of the reviewers. Suppose the refund-agent sees $\lambda_{\text{in}} = 120$ requests/hour, of which a fraction $p_{\text{appr}} = 0.25$ land in the APPROVE tier (the rest are auto / notify and never pause). The expected reviewer load is just the thinned arrival rate:

$$\text{load} = \lambda_{\text{in}} \cdot p_{\text{appr}} = 120 \times 0.25 = \mathbf{30} \text{ approvals/hour.}$$

These approvals can *batch* arithmetically (more arrivals share the same reviewers) but — per the module’s “no batch-approve” UX rule — each is still decided one at a time. Now apply **Little’s law** to the pending-approval queue. With approvals arriving at $\lambda = 30/60 = 0.5$ per minute and each spending $W = \mathbb{E}[\text{latency}] = 28.65$ min in the system, the steady-state number of approvals *pending at any instant* is

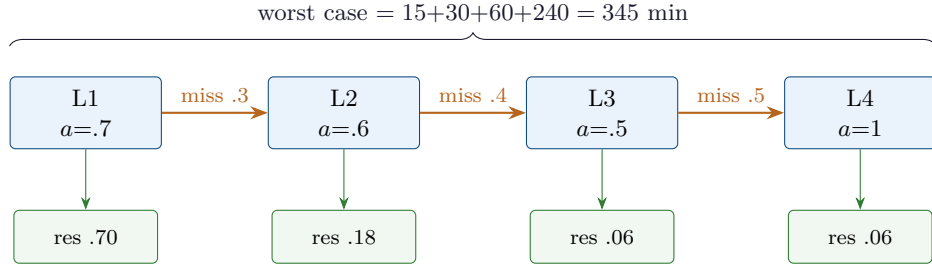
$$L = \lambda W = 0.5 \times 28.65 = \mathbf{14.325} \text{ pending approvals.} \quad (4)$$

That is the number that sizes your review queue UI and your reviewer headcount: at steady state roughly 14 refund approvals sit waiting at once. ✓ If you cannot staff that depth, you have two dials — shrink W (tighter SLAs, more reviewers per rung) or shrink λ (raise the APPROVE threshold so fewer actions gate; that is Topic 2’s scoring model).

Mini-example — this operation in isolation

The *expedited path* is the cheapest dial of all. The module notes high-confidence actions (confidence 0.91 in its queue mock-up) can skip the human entirely. Suppose 40% of approval-tier actions are confident enough to auto-approve. Then the human-borne load drops to $30 \times (1 - 0.40) = \mathbf{18}$ approvals/hour, and L falls in proportion. The ladder still exists for the hard 60%; the easy majority never touches it. *That* is how gates scale sublinearly: you do not add reviewers as traffic grows, you raise the bar for what needs a reviewer at all.

6 The ladder, drawn



Brown arrows are *timeouts* (escalation, weighted by miss-probability); green arrows are *resolutions* (the request leaves the ladder). The green weights are exactly the $P(\text{resolved at } i)$ column, and they sum to 1. The picture is Equation (3).

7 Escalation cheat-sheet

Reach rung i	$r_i = \prod_{j<i} (1 - a_j)$
Resolved at rung i	$P_i = r_i a_i$
Wait if resolved at i	$w_i = \sum_{j<i} \text{SLA}_j + \frac{1}{2} \text{SLA}_i$
Expected latency	$\mathbb{E}[\text{lat}] = \sum_i r_i a_i w_i$
Fraction past L1	$1 - a_1$
Worst-case bound	$\sum_i \text{SLA}_i$
Reviewer load	$\lambda_{\text{in}} \cdot p_{\text{appr}}$
Pending queue depth	$L = \lambda W$ (Little's law)
Expedited load	$\lambda_{\text{in}} p_{\text{appr}} (1 - p_{\text{exp}})$

8 An honest word about these numbers

The answer probabilities (0.7, 0.6, 0.5, 1.0) and SLAs (15/30/60/240 min) are illustrative round numbers, chosen so the probability tree is followable by hand — exactly as the ReAct-cost document used round token sizes no one would ship. Your real a_i depend on your on-call coverage, time of day, and how loud the notification is; your real w_i depend on whether reviewers answer early or at the buzzer (see the **Watch out**). What is *not* illustrative is the structure: reach-probabilities multiply prior misses, expected latency is the total-expectation sum $\sum r_i a_i w_i$, the worst case is the bare sum of SLAs, and the pending queue obeys $L = \lambda W$ no matter what the constants are. Change the numbers and the minutes move; the shape of the ladder does not. That shape — bounded worst case, probability-weighted average, expedited escape hatch — is the thing to carry into every approval workflow you build.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the probability tree, the expected latency, the worst-case bound, and the Little's-law queue depth; change the SLAs, the answer probabilities, or the arrival rate to model your own ladder.

```

1  # escalation_sla.py -- reproduces every number in
2  # "Approval SLAs and Escalation Latency, by Hand".
3
4  names = ["L1", "L2", "L3", "L4"]
5  sla = [15, 30, 60, 240] # minutes: 15m, 30m, 1h, 4h
6  ans = [0.70, 0.60, 0.50, 1.00] # P(answer | reached this rung); L4 eventual = 1.0
7
8  # P(reach rung i) = product of all prior misses
9  reach = []
10 p = 1.0
11 for i in range(4):
12     reach.append(p)
13     p *= (1 - ans[i]) # miss this rung -> reach the next
14
15 # P(resolved AT rung i) = reach * answer
16 resolved = [reach[i] * ans[i] for i in range(4)]
17
18 # cumulative prior SLA = full timeouts already burned before rung i
19 cum_prior = [0]
20 for i in range(1, 4):
21     cum_prior.append(cum_prior[-1] + sla[i - 1])
22
23 # wait if resolved at rung i = burned timeouts + half this rung's window
24 wait_at = [cum_prior[i] + sla[i] / 2 for i in range(4)]
25
26 print("rung SLA ans P(reach) P(res-at) wait(min)")
27 for i in range(4):
28     print(f"{names[i]} {sla[i]:4d} {ans[i]:.2f} {reach[i]:7.4f} "
29           f"{resolved[i]:7.4f} {wait_at[i]:6.1f}")
30 print(f"sum P(resolved) = {sum(resolved):.4f}")
31
32 E_lat = sum(resolved[i] * wait_at[i] for i in range(4))
33 print(f"E[approval latency] = {E_lat:.4f} min")
34
35 frac_past_L1 = 1 - ans[0]
36 print(f"fraction auto-escalated past L1 = {frac_past_L1:.2f}")
37 print(f"P(reach L3) = {reach[2]:.4f} P(reach L4) = {reach[3]:.4f}")
38 print(f"worst-case bounded wait = {sum(sla)} min = {sum(sla)/60:.2f} h")
39
40 # --- throughput / batching / Little's law ---
41 arrival_rate = 120 # requests/hour into the agent
42 p_need_approval = 0.25 # share that land in the APPROVE tier
43 reviewer_load = arrival_rate * p_need_approval
44 print(f"\nreviewer load = {reviewer_load:.1f} approvals/hour")
45
46 lam_per_min = reviewer_load / 60.0
47 L_queue = lam_per_min * E_lat # Little's law: L = lambda * W
48 print(f"lambda = {lam_per_min:.4f} approvals/min")
49 print(f"Little's law queue depth L = {L_queue:.4f} pending")
50
51 p_expedite = 0.40
52 human_load = reviewer_load * (1 - p_expedite)
53 print(f"\nexpedited path skips {p_expedite:.2f} -> "
54       f"human load = {human_load:.1f} approvals/hour")
55
56 # Expected output:
57 # rung SLA ans P(reach) P(res-at) wait(min)
58 # L1 15 0.70 1.0000 0.7000 7.5
59 # L2 30 0.60 0.3000 0.1800 30.0
60 # L3 60 0.50 0.1200 0.0600 75.0
61 # L4 240 1.00 0.0600 0.0600 225.0
62 # sum P(resolved) = 1.0000
63 # E[approval latency] = 28.6500 min

```

```
64 # fraction auto-escalated past L1 = 0.30
65 # P(reach L3) = 0.1200 P(reach L4) = 0.0600
66 # worst-case bounded wait = 345 min = 5.75 h
67 #
68 # reviewer load = 30.0 approvals/hour
69 # lambda = 0.5000 approvals/min
70 # Little's law queue depth L = 14.3250 pending
71 #
72 # expedited path skips 0.40 -> human load = 18.0 approvals/hour
```

— *end of document* —